

DEVOPS · SRE · CLASS EDITION

- **Monitoring**
- **Observability**
- **SRE**

## Hands-On Lab Guide

Why we monitor · how to run the lab · how to break it · how to talk about it in an interview

*No deep IT background required. Follow along in class.*

• **12**

failure modes you can inject

• **10**

scripted incident scenarios

• **2**

chairs: who ships it, who owns the SLO

# How to use this deck

Four parts. Read one, do one, break one, practise one.

1

## Concepts

Plain-language ideas: why we monitor, golden signals, SLOs, error budgets, alerts, and who gets paged.

*Read first, before you touch the cluster.*

2

## The Lab

Build the cluster, deploy the apps and monitoring, and write down what healthy looks like.

*Do this with the cluster in front of you.*

3

## Breaking it

Twelve failure modes and ten scripted incidents. Find each one in Datadog before you reach for kubectl.

*The part that actually teaches.*

4

## Interview

Turn what you did into answers a hiring manager wants to hear — from either chair.

*Practise out loud with a partner.*

## Companion documents in the repo

student-guide.md · testing-the-lab.md · devops-vs-sre.md · slo-sla-sli.md · error-budget.md · architecture.md · runbooks/ · incident-scenarios/

## PART 1

# Concepts

The ideas behind every dashboard, alert and incident you will meet in the lab.



# Why monitoring exists

It is not because servers look interesting.

Someone outside the company is trying to use the product.

If they cannot finish, the business has a problem, even if the CPU graphs look fine.

## That person might be:

● buying something online

● transferring money

● checking grades

● filing a support ticket

## SIMPLE RULE

We monitor so we know when real users cannot do the thing they came to do.

## INTERVIEW ONE-LINER

"I set up monitoring based on whether customers can successfully use the product, not only on whether servers look busy."

# Monitoring vs observability

You need both. They answer different questions.

## ● Monitoring

Watching for known bad conditions and telling you about them.

*"Error rate on checkout is above 5% for 10 minutes. Page the on-call."*

**It wakes you up.**

## ● Observability

Being able to ask new questions when something unfamiliar breaks.

*Metrics + logs + traces together, so you can follow a problem you have never seen before.*

**It tells you why.**

An alert that fires with no way to investigate is a smoke alarm with no lights on in the house.

# Metrics, logs and traces

Real incidents start with a complaint or an alert, never with "p95 is bad".

## ● Metrics

Summary numbers over time: request count, error %, how slow.

**Confirm how big the problem is.**

## ● Traces

One request followed through the system, step by step.

**Find which step is slow or failing.**

## ● Logs

The detailed messages the application wrote.

**Get the exact error, once you know where to look.**

### The usual order



**In this lab:** Datadog carries all three. dd-trace links a trace from the browser through the backend to the Postgres query that was slow.

# The four golden signals

Ask these four questions of every important service.

## 1 Traffic

How many requests are we getting?

requests per second, by endpoint

## 2 Errors

What fraction of them are failing?

5xx rate, failed checkouts

## 3 Latency

How slow are the slower typical requests?

p95 response time (not average)

## 4 Saturation

Are we close to a resource limit?

CPU, memory, restarts, queue depth

### WHY NOT JUST AVERAGES?

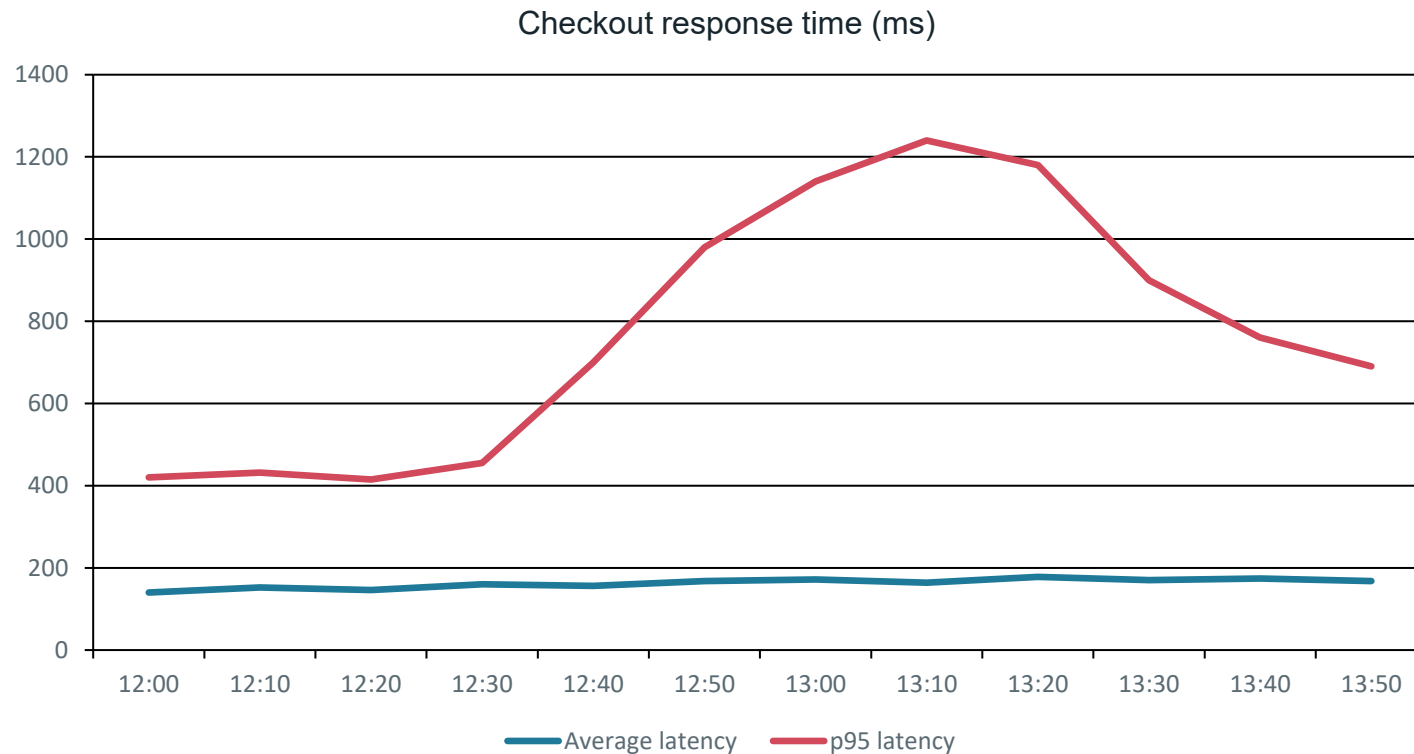
An average hides a bad tail. Most users can be fine while a large group is very slow, and the mean barely moves.

p95 answers:

"For the slower typical requests, how bad is it?"

# What an average hides

Same service, same afternoon, two very different stories.



The average says

**"fine"**

It moved from 140 ms to 175 ms. Nobody would page for that.

The p95 says

**"1 in 20 users waited 1.2 s"**

That is the group filing tickets and abandoning carts. Alert on the tail.



# SLI, SLO, SLA

Measure it, set a bar, promise it to the customer.

**SLI**

## Service Level Indicator

What you measure.

*"Can users complete a purchase successfully and quickly?"*

**SLO**

## Service Level Objective

Your internal goal for that measurement.

*"99.5% of purchases complete under 500 ms, measured over 30 days."*

**SLA**

## Service Level Agreement

The external promise to customers, often with credits attached.

*Usually set looser than the SLO, so you have room to react.*

SLI = measure · SLO = our bar · SLA = customer contract

In an incident people only say "buying is broken." Mapping that onto the SLI is what makes the impact measurable.

# Error budget

If 99.5% must be good, then 0.5% is allowed to be bad. That allowance has a name.

```
# 30-day window, 2,000,000 checkout requests
SLO          = 99.5% good
allowed bad  = 0.5% x 2,000,000 = 10,000 requests

# day 18: 6,200 requests already bad or too slow
budget spent = 6,200 / 10,000 = 62%
```

## ● Budget healthy

Ship features. Take calculated risks. Deploy on a Friday if you want.

## ● Budget burning fast

Slow risky changes. Spend the sprint on reliability instead.

Error budget, day 18 of 30



Error budgets turn monitoring into a shared decision between engineering and product, not just another page.

# Good alerts vs noisy alerts

Every alert should cost somebody sleep only if it deserves to.

## ● Good alert

- A human should act on it
- Tied to customer impact or fast budget burn
- Has a clear first-check path, a runbook
- Fires rarely enough that people still trust it

## ● Noisy alert

- Fires on a small blip no user noticed
- Pages so often the team mutes the channel
- Nobody knows what to do when it arrives
- Measures a server, not an outcome

If customers notice outages before the team does, the alerts are not good enough yet.

*In this lab: four monitors ship with it — high error rate, high latency, pod restarts and memory saturation. Part 3 shows you three failure modes none of them can see.*

# Monitoring Kubernetes

Two layers, and one trap that catches almost everyone.

## ● Platform layer

*Is the cluster able to run the workload?*

- Nodes: ready, capacity, pressure
- Container restarts and OOMKill events
- Pods that cannot be scheduled
- Rollouts that never became healthy

*Example monitor: more than three restarts in ten minutes.*

## ● Application layer

*Is each service serving users?*

- Traffic per service and endpoint
- Error rate per service
- p95 latency per service
- Dependency health (database, cache, queue)

*Example monitor: 5xx rate above 5% for five minutes.*

## THE TRAP

Pod status says Running, and users still fail. Readiness must fail when a hard dependency is unreachable. In this lab /readyz runs a real SELECT 1, so a pod that loses Postgres leaves the Service.

## Tag everything consistently

env · service · version

It is what joins metrics, logs and traces — and it must reach cluster-level data too.

**Check both layers, in this order:** kubectl get pods → describe pod → service dashboard → trace → logs

# Incidents, runbooks, postmortems

The three words that show up in every SRE job description.

## ● Incident

Something is wrong for users, or is about to be.

## ● Runbook

A short checklist of what to check first for a known failure mode.

## ● Postmortem

What we write afterwards so the next person is faster than we were.

Runbooks for this lab live in `docs/runbooks/`

## The postmortem, in six lines

- 1 How we found out: alert, support ticket, or user report
- 2 What customers could not do
- 3 How we diagnosed it
- 4 What actually fixed it
- 5 Impact, including error budget if you can estimate it
- 6 One prevention improvement

***Blameless: fix the system and the detection, never the person.***

# Who actually gets paged first?

It depends on the operating model, not on the job title.



## Dedicated SRE team

SRE holds the primary pager for services they have onboarded. The development team is secondary. Common where formal error budgets exist.

**First responder: SRE**



## You build it, you run it

The team that wrote the service carries its own pager. No separate first line; the platform team is a supplier, not a responder.

**First responder: the service team**



## Platform team owns the plumbing

The usual shape where the title is DevOps and there is no SRE function. The pager covers cluster, pipeline and infrastructure.

**First responder: split by layer**

## The rule that survives every org chart

Ownership follows the layer, not the title. Where there is no SRE function, the DevOps engineer is page number one for the cluster, the pipeline and the infrastructure — and that is triage.

## DO NOT SAY THIS IN AN INTERVIEW

"Triage is not my job." Say instead: "I own the platform layer and the observability the service teams depend on. For an application incident I am the escalation path, not the first responder, and I can still read the trace."

# Two chairs, one incident

Titles vary by company. Ownership follows the layer — and the two chairs ask different first questions about the same facts.

|                | <div>● DevOps engineer</div>                                                            | <div>● SRE</div>                                                               |
|----------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Owns           | The pipeline and the platform: build, image, manifest, config, secret, ingress, cluster | The service level: what “working” means, and whether detection was good enough |
| First question | “What changed?”                                                                         | “What did it cost, and who told us?”                                           |
| Pages on       | Rollouts that never went healthy, nodes, capacity, the delivery path                    | Symptoms with user impact: error rate, p95, availability                       |
| Fixes with     | rollout undo, a corrected ConfigMap or Secret, a pipeline guardrail                     | A rollback decision, then a durable fix to whatever let it through             |
| Closes with    | The change reverted, and a pipeline that can't ship it again                            | Budget spent, a named detection gap, one prevention item with an owner         |

Where there is no SRE function — which is most places advertising a DevOps role — you sit in both chairs. The second one is the half people forget to practise.

# Did something change, or did something drift?

The first branch in any real triage. It decides the fix, the owner, and the shape of the next hour.

## Change-driven

Somebody shipped: a config value, a rotated credential, an ingress edit, a limit, a bad image tag.

**Clock starts** at the merge, not the alert

**Evidence** in rollout history and events

**Fastest fix** is almost always to put it back

**Often silent** — old pods keep serving

## Condition-driven

The system met a limit nobody moved: a slow dependency, a leak, a pool exhausted by a neighbour, a spike.

**Nothing shipped** — there is nothing to undo

**Evidence** in traces, saturation, dependencies

**Fix** is mitigation, then the root cause

**Usually loud** — errors, latency, restarts

```
# ten seconds, and it tells you which family you are in
kubectl -n <app> rollout history deployment/<app>-backend
kubectl -n <app> get events --sort-by=.lastTimestamp | tail -20
```

A rollout minutes before the symptom? Revert first, explain afterwards. Last rollout days ago? Stop hunting for a culprit change.



# How a real investigation sounds

Use this order when you speak, when you write, and when you are interviewed.

|   |                                     |                                                                      |
|---|-------------------------------------|----------------------------------------------------------------------|
| 1 | <b>How did we know?</b>             | Alert, support ticket, or a user report. Say this first.             |
| 2 | <b>What could customers not do?</b> | In their words: "checkout failed", not "the pod restarted".          |
| 3 | <b>What did we check first?</b>     | Name the dashboard or the runbook step you opened.                   |
| 4 | <b>What did we discover?</b>        | Errors up, responses slow, and which step in the trace was to blame. |
| 5 | <b>What did we change?</b>          | The fix, plus the graph that came back down.                         |

## DO NOT OPEN WITH

"p95 was elevated..."

Jargon first sounds rehearsed. Say how you were alerted and what users felt, then bring the numbers.

## SAY THIS INSTEAD

"Support saw checkout failures at 2pm. Our error-rate monitor had fired four minutes earlier, so we opened the checkout dashboard first."

## PART 2

# Doing the lab

Build it, watch it, and write down what healthy looks like.



# Before you start

Get all of this in place first. The walkthrough assumes it is ready.



## Local tools

`terraform · aws · kubectl · docker · helm · jq · envsubst`



## bash 4.4 or newer

`macOS ships 3.2 -- brew install bash, then run setup.sh with it`



## Your own AWS account

Real EKS, RDS and NAT. No free tier, and no sharing.



## A Route 53 public hosted zone

Same account, a domain you control, NS delegation pointing at it.



## Your own free Datadog trial

Nothing in the repo contains a real API key.

## ONE DEPLOYMENT PER AWS ACCOUNT

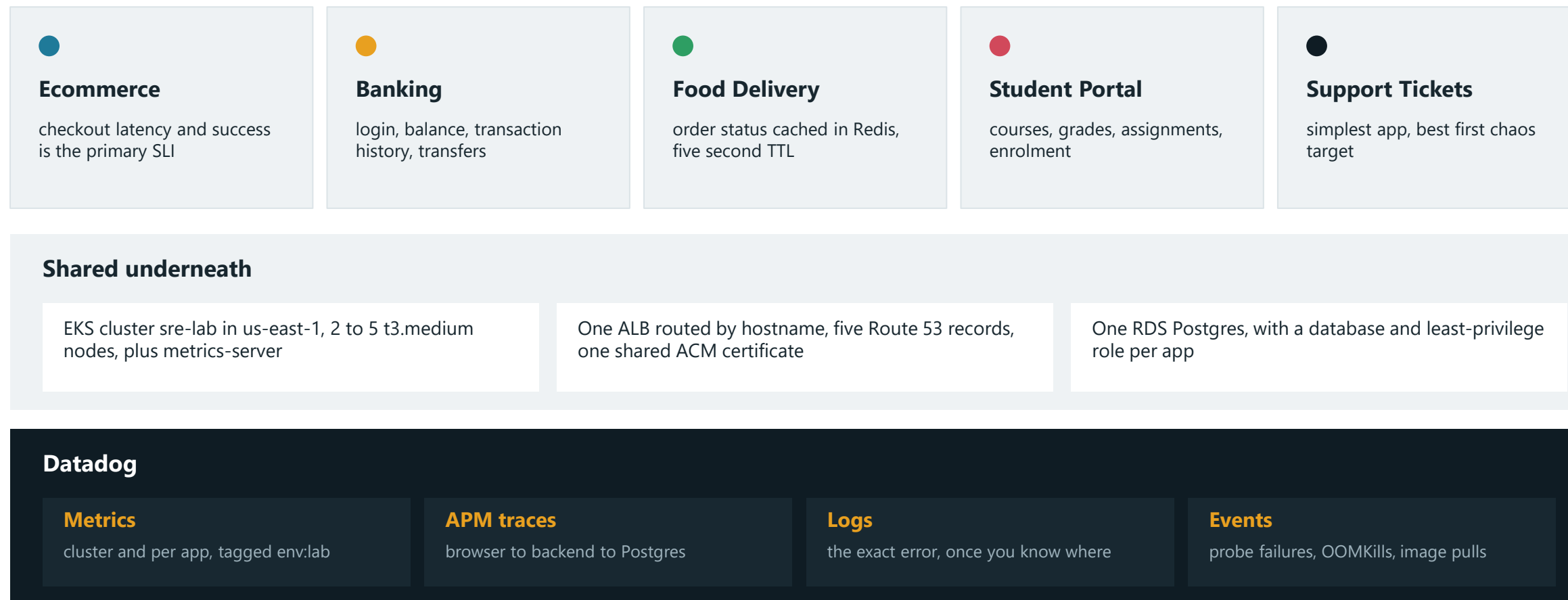
Resource names are fixed, not parameterised per student: the sre-lab cluster, the RDS instance, the ECR repositories. Two students in one account will collide.

## It costs real money

Roughly \$150 to \$250 a month if left running  
EKS, 2 to 5 t3.medium nodes, a NAT gateway, RDS  
Run `teardown.sh` at the end of every session

# What you are building

Five production-style applications on one shared cluster, observed with Datadog.



# Steps 1 and 2: your Datadog account, then your domain

Each student uses their own trial. There is no shared account.

|   |                           |                                                     |
|---|---------------------------|-----------------------------------------------------|
| 1 | Start a free trial        | datadoghq.com                                       |
| 2 | Copy your API key         | Organization Settings > API Keys. Required.         |
| 3 | Copy your application key | Organization Settings > Application Keys. Optional. |
| 4 | Note your site            | The <site> in app.<site>, e.g. us5.datadoghq.com    |

```
cp terraform/terraform.tfvars.example terraform/terraform.tfvars
# then set dns_zone_name to your domain
```

*The lab looks the zone up by name and will not create one for you. It adds five records under it, and finds or requests one ACM certificate.*

## THE CLASSIC MISTAKE

Trials are not all on the same site. Get the site wrong and the Agent pods still come up Running, silently fail to authenticate, and nothing ever appears in your dashboard. Check the site value before you move on.

## Without the application key

setup.sh still installs the Agent. Metrics, APM, logs and events all work, and RUM survives via the cached .rum-apps.json. Only the dashboard and monitor import is skipped.

# Step 3: deploy everything with setup.sh

One command. Expect 15 to 20 minutes, mostly waiting on EKS and the ALB.

```
DATADOG_API_KEY=<your-datadog-api-key> \  
DATADOG_APP_KEY=<your-datadog-app-key> \  
DATADOG_SITE=<your-site, default datadoghq.com> \  
./scripts/setup.sh
```

## It prints all five app URLs at the end

Real Route 53 names that work immediately. No hosts-file editing anywhere in this lab.

## Reconnecting kubectl in a new shell

```
aws eks update-kubeconfig --name sre-lab --region us-east-1
```

Also the fix when Docker Desktop hijacks your context.

## Datadog is optional here

Omit all three DATADOG\_ variables to skip Datadog for now, then add it later by re-running the same command with them set. Everything else is safe to re-run.

## THE IMPORT IS NOT IDEMPOTENT

Re-running the dashboard import creates duplicates. Only pass DATADOG\_APP\_KEY on the runs where you actually want to import. Monitors update by name; dashboards do not.

# What setup.sh actually does

Worth reading while it runs. Being able to describe this is an interview answer in itself.

|                                                                                                                                 |                          |                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------|--------------------------|--------------------------------------------------------------------------------------------|
| 0                                                                                                                               | <b>Preflight</b>         | CLIs, Docker, AWS creds, bash 4.4+, the hosted zone, NS delegation, and an ACM certificate |
| 1                                                                                                                               | <b>terraform apply</b>   | VPC, EKS + node group, the shared RDS instance, 10 ECR repositories                        |
| 2                                                                                                                               | <b>update-kubeconfig</b> | Points kubectl at the cluster                                                              |
| 3                                                                                                                               | <b>RUM apps</b>          | Provisions or reuses one Datadog RUM application per app                                   |
| 4                                                                                                                               | <b>Build and push</b>    | All 10 images to ECR, tagged with a timestamp                                              |
| 5                                                                                                                               | <b>Namespaces</b>        | Five, each with its own ResourceQuota and LimitRange                                       |
| 6                                                                                                                               | <b>Databases</b>         | Per app: a database, a least-privilege role, init.sql, a Secret                            |
| 7                                                                                                                               | <b>Redis</b>             | Food Delivery gets its in-cluster cache                                                    |
| 8                                                                                                                               | <b>Deploy + verify</b>   | Applies every manifest, then blocks until all 10 rollouts are healthy                      |
| 9                                                                                                                               | <b>Ingress</b>           | metrics-server, the ALB controller, Ingress, then the DNS records                          |
| 10                                                                                                                              | <b>Datadog</b>           | Agent and Cluster Agent, plus dashboards and monitors if the app key is set                |
| <b>Step 8 is new:</b> a deploy that never became healthy used to exit 0 and leave an app on the old image. Now it fails loudly. |                          |                                                                                            |

# Step 4: verify the deployment

Five URLs, five real user actions, then confirm that tracing works end to end.

## All five share one ALB, routed by hostname

```
https://ecommerce.<your-domain>  
https://banking.<your-domain>  
https://food-delivery.<your-domain>  
https://student-portal.<your-domain>  
https://support-tickets.<your-domain>
```

```
cat .lab-domain  
echo "https://ecommerce.$(cat .lab-domain)"
```

*The domain suffix is saved to .lab-domain, and the chaos scripts read it automatically.*

## Complete one real user action in each app

- |                          |                                   |
|--------------------------|-----------------------------------|
| ● <b>Ecommerce</b>       | add to cart and check out         |
| ● <b>Banking</b>         | log in, check a balance (demo123) |
| ● <b>Food Delivery</b>   | place an order                    |
| ● <b>Student Portal</b>  | view grades                       |
| ● <b>Support Tickets</b> | submit a ticket                   |

Then open APM > Traces. A trace should span browser → backend → Postgres. If it starts at the backend, RUM didn't load — check window.DD\_RUM in the console.



# Step 5: observe the healthy baseline

You cannot recognise abnormal until you have written down normal.

## Open these while everything is healthy

Each app dashboard, from [datadog/dashboards/](#)  
The SRE Lab Overview dashboard  
[docs/slo-sla-sli.md](#), for what each one measures

## The instructor question

"Which widget moved first, and by how much?"

You can only answer that if you looked at the dashboard before anything was broken.

## Baseline worksheet

Write these down now, per app.

|                |                     |       |
|----------------|---------------------|-------|
| <b>Traffic</b> | requests per second | _____ |
|----------------|---------------------|-------|

|               |                    |       |
|---------------|--------------------|-------|
| <b>Errors</b> | percentage failing | _____ |
|---------------|--------------------|-------|

|                |                            |       |
|----------------|----------------------------|-------|
| <b>Latency</b> | p95 in ms, not the average | _____ |
|----------------|----------------------------|-------|

|                   |                                |       |
|-------------------|--------------------------------|-------|
| <b>Saturation</b> | memory against the 256Mi limit | _____ |
|-------------------|--------------------------------|-------|

|                 |                        |       |
|-----------------|------------------------|-------|
| <b>Restarts</b> | count before you start | _____ |
|-----------------|------------------------|-------|

## PART 3

# Breaking it on purpose

Twelve failure modes, ten scripted incidents, and one loop that works for all of them.



# The experiment loop

The same six steps for every failure mode. The order is the skill.

|   |                                       |                                                                                     |
|---|---------------------------------------|-------------------------------------------------------------------------------------|
| 1 | <b>Baseline</b>                       | Look at the dashboard. Before, not after.                                           |
| 2 | <b>Inject</b>                         | <code>./scripts/chaos/&lt;script&gt;.sh &lt;app&gt;</code>                          |
| 3 | <b>Datadog first</b>                  | How big, and who? Then logs and events for the exact error.                         |
| 4 | <b>Confirm as a customer</b>          | <code>curl</code> the real URL. Does it match what users report?                    |
| 5 | <b>Only now, <code>kubectl</code></b> | READY column, describe pod, rollout history.                                        |
| 6 | <b>Undo, and prove it</b>             | <code>reset.sh</code> or <code>--undo</code> , then re-check the widget that moved. |

## Every app runs 2 replicas

A script's single `curl` goes through the ALB to ONE random pod, so effects are partial by design. Port-forward to a pod to be deterministic.

If you opened a terminal before you opened a dashboard, you skipped the exercise.

# Twelve failure modes, three families

Thirteen scripts live in `scripts/chaos/` — `reset.sh` is the undo tool, not a fault.

## A · In-app conditions

Toggled over HTTP against a live pod. No redeploy.

`inject-latency.sh`

`inject-errors.sh`

`memory-spike.sh`

`cpu-spike.sh`

`drop-db-connection.sh`

Undo: `reset.sh`

## B · Kubernetes-level

Bypass the app entirely.

`kill-random-pod.sh`

`scale-to-zero.sh`

Undo: `kubectl`

## C · Change-driven

Somebody shipped something. Deliberately quiet.

`bad-deploy.sh`

`break-config.sh`

`rotate-secret.sh`

`break-ingress.sh`

`shrink-limits.sh`

Undo: `--undo`

**Family C is new, and it is the DevOps half of the lab.** In most of those, customers stay fine while the service quietly loses its ability to heal.

# A1 · Slow, but not failing

```
./scripts/chaos/inject-latency.sh ecommerce 4000
```

## What you should see

p95 climbs hard  
Error rate stays completely flat  
The average barely moves

## Undo

```
./scripts/chaos/reset.sh ecommerce
```

## Where to look

The p95 widget — compare it with the average on purpose  
curl https://ecommerce.<domain>/api/chaos shows latencyMs: 4000

## The point

Slow is not failing. An error-rate monitor never fires here, which is why latency needs its own alert.

## A2 · Failing fast

```
./scripts/chaos/inject-errors.sh banking 0.3
```

### What you should see

Error rate jumps  
Traces flagged as errors  
5xx filling the logs

### Where to look

Dashboard: the Error Rate widget  
Logs: service:banking-backend status:error

### Undo

```
./scripts/chaos/reset.sh banking
```

### The point

Errors and latency are two signals with two different responses. Don't collapse them into "it's broken".

## A3 · The leak that heals itself

```
./scripts/chaos/memory-spike.sh support-tickets 300
```

### What you should see

Memory climbs toward the 256Mi limit  
A real OOMKilled, then a restart  
Then it looks fine again

### Where to look

describe pod | grep -A5 "Last State" → Reason: OOMKilled  
Dashboard: Pod Restarts, and memory just before the kill

### Undo

Nothing — the restart cleared the in-memory state.

### The point

Self-resolving incidents get waved off. The saturation monitor exists to fire BEFORE the kill.

# A4 · Saturation, and the HPA

```
./scripts/chaos/cpu-spike.sh ecommerce 60
```

## What you should see

Latency spikes on every endpoint that pod serves  
The HPA scales out past 2 replicas

## Where to look

kubectl -n ecommerce get hpa -w (start this first, in a second terminal)  
Dashboard: p95 across all endpoints, not just one

## Undo

Ends by itself; the HPA scales back after cooldown.

## The point

Saturation shows up as latency everywhere, not as errors on one endpoint.



# A5 · Running, but not Ready

```
./scripts/chaos/drop-db-connection.sh student-portal
```

## What you should see

/readyz returns 503; /healthz stays 200

Pod goes NotReady after ~30s and leaves the Service

Nothing restarts — the process never crashed

## Where to look

Logs: "readiness check failed: db connection dropped (chaos)"

kubectl get endpoints — the pod is gone from the list

## Undo

kubectl exec -n <ns> <pod> — node -e "...api/chaos/reset..." (reset.sh can't reach it)

## The point

The most-asked interview question. Liveness and readiness gate different things.

# B1 and B2 · Bypassing the app

```
./scripts/chaos/kill-random-pod.sh banking  
./scripts/chaos/scale-to-zero.sh support-tickets support-tickets-frontend
```

## B1 · kill-random-pod

Restart count +1, a brief blip if you're watching.

**Undo** self-heals — the Deployment reschedules immediately

**The point** alert on restart COUNT over a window, not one restart

## B2 · scale-to-zero

Full outage. The ALB returns 503 — no targets at all.

**Undo** `kubectl scale deployment/... --replicas=2`

**The point** compare this 503 with C4's 404: an empty target group and an unhealthy one behave differently

# C1 · The tag that never existed

```
./scripts/chaos/bad-deploy.sh student-portal student-portal-backend student-portal-backend
```

## What you should see

New pods ImagePullBackOff

Old pods keep serving

So the symptom is INTERMITTENT, not an outage

## Where to look

describe pod → "Failed to pull image ... not found"

rollout status → stuck waiting for replicas that never come

## Undo

```
kubectl -n student-portal rollout undo deployment/student-portal-backend
```

## The point

The partial outage is RollingUpdate doing its job — it won't kill healthy pods for replacements that aren't ready.

## C2 · The config that shipped fine

```
./scripts/chaos/break-config.sh banking
```

### What you should see

One new pod never becomes Ready, then CrashLoops  
The rollout stalls  
The app still returns 200 — customers are fine

### Where to look

Events: "Readiness probe failed ... connection refused" on :4000  
logs <newest pod> → "listening on 4001" ← the penny drops

### Undo

```
./scripts/chaos/break-config.sh banking --undo
```

### The point

rollout undo does NOT fix this: the pod template never changed, only the ConfigMap it references.

## C3 · Rotated in one place

```
./scripts/chaos/rotate-secret.sh ecommerce
```

### What you should see

New pod Running but 0/1, with ZERO restarts  
No monitor fires at all  
Endpoints list 2 addresses, not 3

### Undo

```
./scripts/chaos/rotate-secret.sh ecommerce --undo
```

### Where to look

Logs: "password authentication failed for user ecommerce\_app"  
port-forward, then curl /healthz (200) vs /readyz (503)

### The point

The working credential now exists only in the memory of two pods that can't be replaced. Zero impact, high urgency.

## C4 · Green dashboards, angry customers

```
./scripts/chaos/break-ingress.sh food-delivery
```

### What you should see

A 404 from the BACKEND — not a gateway error  
X-Powered-By: Express in the headers  
/api/\* still returns 200  
Every dashboard green; backend throughput RISES

### Undo

```
./scripts/chaos/break-ingress.sh food-delivery --undo
```

### Where to look

```
curl -i https://food-delivery.<domain>/ | head -12  
aws elbv2 describe-target-health ... (target health is in AWS, not Datadog)
```

### The point

A total outage your monitoring cannot see, because every instrument sits inside a service that isn't broken.

# C5 · The ceiling that moved

```
./scripts/chaos/shrink-limits.sh support-tickets
```

## What you should see

New pods OOMKilled during startup (exit 137)  
CrashLoopBackOff; old pods keep serving  
Identical symptom to A3 — opposite cause

## Where to look

rollout history, and compare the resources block with another app  
Dashboard: memory usage never grew. The LIMIT moved.

## Undo

```
./scripts/chaos/shrink-limits.sh support-tickets --undo
```

## The point

rollout undo DOES work here (the pod template changed), unlike C2 and C3.  
Explaining that difference is worth more than the fix.

# The detection gap

Three of those twelve are invisible to every monitor the lab ships.

## What nothing pages on today

A pod that is Running but never Ready

A rollout that stalled and will never finish

Throughput falling to zero

A deploy that reported success and shipped nothing

## Add: unavailable replicas

```
max(last_10m):max:kubernetes_state.deployment  
  .replicas_unavailable{env:lab} by {kube_deployment} > 0
```

Bad image, failed probe, OOMKill loop, quota-blocked surge — four root causes, one query, and it fires while old pods still serve.

Neither ships with the lab on purpose. Building one of them out of your own postmortem's action item, then watching it fire, is the exercise.

**Verified on a live cluster:** during C4 the backend throughput RISES, because the ALB health-checks it. A throughput monitor would not catch it — only a synthetic check from outside the cluster would.



# The ten scenarios

Each wraps one or more failure modes in a briefing with no answer attached.

| #  | SCENARIO                          | APP                 | STARTS WITH   | USES            |
|----|-----------------------------------|---------------------|---------------|-----------------|
| 01 | The Silent Checkout               | ecommerce           | condition     | A1 + A2         |
| 02 | Payday Panic                      | banking             | condition     | A5              |
| 03 | The Stuck Order                   | food-delivery       | condition     | Redis to 0      |
| 04 | Grades Gone Missing               | student-portal      | <b>change</b> | C1              |
| 05 | The Midnight Memory Leak          | support-tickets     | condition     | A3              |
| 06 | The Noisy Neighbor                | ecommerce + banking | condition     | manual pg_sleep |
| 07 | The 4 PM Config Change            | banking             | <b>change</b> | C2              |
| 08 | The Rotated Password              | ecommerce           | <b>change</b> | C3              |
| 09 | Green Dashboards, Angry Customers | food-delivery       | <b>change</b> | C4              |
| 10 | The Tightened Limit               | support-tickets     | <b>change</b> | C5              |

*The answer key gives every one of them a DevOps walkthrough and an SRE walkthrough. Work them without reading it first.*

# A running order that builds

Roughly 20 to 30 minutes each, with discussion.

|   | RUN                   | WHY HERE                                                           |
|---|-----------------------|--------------------------------------------------------------------|
| 1 | A1 on support-tickets | Simplest app, cleanest signal. Teaches p95 vs average immediately  |
| 2 | A5 on banking         | Running vs Ready — the most-asked interview question               |
| 3 | A3 on support-tickets | A self-healing incident; sets up the contrast for step 5           |
| 4 | C2 on banking         | First change-driven one: “what changed?” before “what's broken?”   |
| 5 | C5 on support-tickets | Same OOMKill as step 3, opposite cause. The contrast IS the lesson |
| 6 | C4 on food-delivery   | Finish here: total outage, every dashboard green                   |

# Runbook example: the first ten minutes

Short, ordered, and written to be read by someone who did not build the service.

docs/runbooks/high-latency.md

**TRIGGER**

Ecommerce checkout p95 above 2s for 5 minutes

**SEVERITY**

SEV2

**USER IMPACT**

Checkout hangs, carts get abandoned

**PRIMARY**

Ecommerce service owner

**SECONDARY**

Platform on call

**DASHBOARD**

SRE Lab · Ecommerce

|   |                                 |                                                          |
|---|---------------------------------|----------------------------------------------------------|
| 1 | Confirm real users are affected | Open the checkout widget, then try the flow yourself.    |
| 2 | Scope it                        | One endpoint or all of them? One pod, or every pod?      |
| 3 | Go to APM before kubectl        | Which span owns the time: browser, backend, or Postgres? |
| 4 | Check the dependencies          | RDS connections first, then Redis if the app uses it.    |
| 5 | Read one slow trace end to end  | Then open the log line for that trace id. Do not guess.  |

# Runbook example: decide, escalate, close

The part most runbooks are missing: what to do once you know something.

- **A recent change or an injected fault explains it**

Undo it first, diagnose afterwards. Re-check p95 within five minutes.

```
./scripts/chaos/reset.sh ecommerce
```

- **A dependency is unhealthy**

If /readyz is failing, the pod has already left the Service. Check RDS connections and the credentials Secret.

- **Neither, fifteen minutes in**

Escalate to the service owner and declare an incident. Do not keep digging on your own.

## DO NOT

Delete the pod before you have captured a slow trace.  
You delete the evidence.

Silence the monitor.

Fix it and walk away without writing down the timeline.

## EXIT CRITERIA

Checkout p95 back to baseline for ten minutes, and the flow works end to end in the browser. Only then close it and start the postmortem.

# Find it in Datadog, then fix it

Dashboard first, kubectl second. The order is the whole skill.

|   |                                  |                                                                                               |
|---|----------------------------------|-----------------------------------------------------------------------------------------------|
| 1 | <b>How did we know?</b>          | A monitor fired, or the app felt wrong. Write down the time — you need it for the postmortem. |
| 2 | <b>What could users not do?</b>  | Say it the way a customer would: checkout hangs, not the pod restarted.                       |
| 3 | <b>Which widget moved first?</b> | And by how much? Compare against your baseline. This is the graded question.                  |
| 4 | <b>Only now, kubectl</b>         | get pods, describe, logs. Try the commands before opening the runbook.                        |
| 5 | <b>Fix it and prove it</b>       | Confirm recovery in the app itself and on the dashboard. Metrics lag a minute or two.         |

*If you opened a terminal before you opened a dashboard, you skipped the exercise.*

# Write the postmortem

Fill this in while the incident is still fresh. Half a page is enough.

- **Detection**

Which monitor fired, at what time, and how you first heard about it.

- **Diagnosis**

The path you took: dashboard, then trace, then log line.

- **Error budget burned**

Use the method in docs/error-budget.md.

- **Customer impact**

What users could not do, for how long, and roughly how many.

- **Fix**

What you changed, and the evidence it worked.

- **One prevention item**

A better alert, a probe, a limit. One thing, with an owner.

Blameless means the write-up names systems and gaps in detection, never a person who should have noticed.

# Postmortem example: slow checkout

Half a page, written the same afternoon.

## Summary

Ecommerce checkout slowed from 400ms to about 3s at p95. Nothing errored, so the error-rate monitor stayed quiet and the average latency widget barely moved. Cleared 23 minutes after it started.

**23 min**

degraded

**~400**

slow checkouts

**4%**

of the monthly budget

Detected by the latency monitor, six minutes before the first complaint.

## Timeline

- 13:58** latency fault injected on the ecommerce backend
- 14:02** checkout p95 monitor fires at 3.1s
- 14:06** on call opens the Ecommerce dashboard
- 14:09** average latency looks fine; p95 is four times baseline
- 14:14** APM shows the delay inside the backend span, not Postgres
- 14:18** fault cleared with chaos/reset.sh
- 14:21** p95 back under 500ms
- 14:31** incident closed, postmortem started

# Postmortem example: cause and follow-up

Systems and gaps in detection, never a name.

## ● Root cause

A delay was introduced ahead of every response on the ecommerce backend. In production this is the same shape as a slow dependency or a bad release.

## ● Why the alerts stayed quiet

Nothing returned an error, so the error-rate monitor had nothing to fire on. Slowness is not failure, and it needs its own alert.

## ● Why it took seven minutes to see

The first widget we opened showed average latency, which barely moved. The p95 widget told the real story straight away.

## Action items

- 1 Page on checkout p95, not on average **ecommerce**
- 2 Put p95 first on the dashboard **ecommerce**
- 3 Alert on unavailable replicas **platform**
- 4 Add a synthetic check from outside the cluster **platform**



# Repeat, then tear it down

Go back and pick another scenario. When you are done for the day, destroy it.

```
./scripts/teardown.sh
```

## What it does, in this order

- Deletes the Ingress resources first, so the shared ALB is released cleanly
- Uninstalls Datadog
- Deletes the five app namespaces
- Runs terraform destroy

## Order matters here too

Load balancers created by Kubernetes are not managed by Terraform. Destroy the cluster first and the ALB is orphaned: still billing, and invisible in your plan.

## CHECK THE CONSOLE AFTERWARDS

- EC2 load balancers
- NAT gateways and elastic IPs
- ECR repositories
- RDS instances

## The certificate stays

The lab uses an ACM certificate but never owns one, so teardown can't delete a certificate something else on your domain is using. Certificates are free.

# When it goes wrong

The issues students actually hit, and what fixes them.

| SYMPTOM                                                                  | WHAT IS REALLY GOING ON                                                                                           |
|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Nothing ever appears in Datadog</b>                                   | Wrong site value. The Agent runs happily and never authenticates. Re-run with the right DATADOG_SITE              |
| <b>kubectl talks to the wrong cluster</b>                                | Docker Desktop rewrote your context on start. <code>aws eks update-kubeconfig --name sre-lab</code>               |
| <b>Preflight aborts on NS delegation</b>                                 | Your registrar is not pointed at the Route 53 hosted zone. Update the NS records, then re-run                     |
| <b>HPA shows cpu: &lt;unknown&gt;</b>                                    | metrics-server missing. <code>setup.sh</code> step 9 installs it; <code>cpu-spike</code> cannot work without it   |
| <b>setup.sh succeeded but an app is on the old image</b>                 | A rollout stalled — often a namespace quota with no headroom for the surge pod. Step 8 now catches this           |
| <b>A monitor on <code>kubernetes_state.*{env:lab}</code> has no data</b> | Cluster metrics need the global tag, set via <code>datadog.tags</code> in <code>helm-values.yaml</code>           |
| <b>Traces start at the backend</b>                                       | RUM missing from the bundle. Check <code>window.DD_RUM</code> ; the ids are cached in <code>.rum-apps.json</code> |
| <b>reset.sh will not clear a database drop</b>                           | That pod already left the Service, so the reset hit a healthy one. Reset it with <code>kubectl exec</code>        |

## PART 4

# Talking about it

Turning a lab you finished into answers that land — from either chair.



# Questions you will actually be asked

Prepare one concrete lab story for each.

**"How do you decide what to alert on?"**

Start with the user journey and the SLO, not with CPU.

**"Walk me through an incident you handled."**

Use the five-step order. Detection first, jargon last.

**"Tell me about a deploy that broke production."**

The DevOps one. Fix the pipeline, not just the incident.

**"What is an SLO, and how did you pick the number?"**

Name the SLI, the target, the window, and who agreed.

**"What is an error budget for?"**

It decides whether the team ships or slows down. Give the maths.

**"A pod says Running but users report errors."**

Readiness vs liveness, dependency health, then one trace.

Every one of these has an answer sitting in the lab you just ran. Specifics beat theory.

# A model answer

Ninety seconds, five beats, no jargon in the opening line.

1

## Detection

Our error-rate monitor on checkout fired at 14:02, four minutes before the first support ticket.

2

## Impact

Customers could add items to the cart, but the final payment step failed for about a third of them.

3

## First check

I opened the checkout runbook. The first step is the service dashboard.

4

## Finding

Errors up and p95 from 400 ms to 1.2 s. A trace showed the database call timing out; the logs gave the exact error.

5

## Fix and proof

We rolled back the change that shrank the connection pool. Baseline within three minutes, about 4% of the monthly budget.

### WHY IT WORKS

Detection before diagnosis

Impact in the user's words

One number per claim

Ends with proof, not opinion

### AVOID

Opening with "p95 was elevated"

Tool names instead of reasoning

Blaming a teammate or a vendor

No numbers at all

# Same facts. Different question.

A config change stalled a rollout for twenty minutes. Customer impact: zero.

## ● The DevOps version — the delivery path

*"A config change merged at 15:40 and the rollout went out at 15:52. New pods came up Running but never Ready, and the rollout stalled with the old ReplicaSet still serving. The event stream showed readiness probes failing with connection refused, so the container wasn't listening where the probe was looking. The new pod's logs said listening on 4001. I reverted the ConfigMap and rolled; Ready in ninety seconds. The real fix is upstream: nothing in the pipeline checks that a rollout became healthy."*

## ● The SRE version — the service level

*"We were one node recycle away from a full outage for twenty minutes and didn't know it. Impact was zero and we spent no budget, because the old pods kept serving — and every one of our four monitors stayed green. That's the finding. Our alerting is symptom-based, which is the right default, and it's blind to a service running with no way to replace a pod. So the action item wasn't another symptom alert. It was one platform alert on unavailable replicas."*

Notice what the second one does NOT do: it never re-tells the diagnosis. It answers what did it cost, what would have caught it, and what do we change.

# How to read the next slides

Each question hides two or three concepts. The rubric is roughly how the answer gets scored.

## **WEAK**

They stop listening. Usually a textbook definition with no experience behind it.

## **SOLID**

You pass. The concepts are right and there is a real example attached.

## **STRONG**

They start asking follow-ups, because you said something worth pulling on.

### **Aim for solid, let them pull you to strong**

Do not deliver the strong answer as a monologue. Give the solid version, stop talking, and let the follow-up take you deeper.

### **THE NUMBERS HAVE TO BE YOURS**

Using this lab as your example is fine, and saying it was a lab is honest. What is not fine is quoting a p95 or a budget figure you cannot defend.

# "How do you decide what to alert on?"

Opening question in almost every SRE and platform interview.

## What is really being tested

Do you start from the user, or from the machine?

Do you know the difference between a symptom and a cause?

Are you willing to say no to an alert someone asked for?

## SAY SOMETHING LIKE

"I start from the user journey. For checkout the question is whether people can pay, so the alert is on checkout success rate and p95 at the payment step, not on CPU. Cause metrics stay on the dashboard for diagnosis. Before an alert goes live it has to pass two tests: a human has to be able to do something about it, and there has to be a runbook. If it fails either one it becomes a dashboard panel, not a page."

## WEAK

Lists CPU, memory and disk, with no link to anything a customer can feel.

## SOLID

Names the golden signals and alerts on errors and latency rather than resources.

## STRONG

Ties the alert to an SLO, separates symptom from cause, and deletes alerts nobody acts on.



# "Walk me through an incident you handled."

The full five-beat version is on the model answer slide. This is what sits underneath it.

## What is really being tested

Can you tell a story in a fixed order under mild pressure?

Do you lead with detection and impact, or with jargon?

Do you close with evidence, or with an opinion?

## SAY SOMETHING LIKE

"Our checkout error-rate monitor fired at 14:02, four minutes before the first support ticket. Customers could add to the cart but the payment step failed for about a third of them. I opened the runbook, checked the dashboard, and a trace showed the database call timing out. We rolled back the change that had shrunk the connection pool; error rate was back to baseline within three minutes. It cost about 4% of the monthly error budget, and the follow-up was a readiness probe that actually queries the database."

## WEAK

Opens with a tool or a metric. No timeline, no customer, no numbers.

## SOLID

Detection, impact, diagnosis and fix, in that order, with at least one real number.

## STRONG

Adds the proof it was fixed, the budget cost and the prevention item. Blames no one.

# "Tell me about a deploy that broke production."

The DevOps counterpart. It tests whether you own the delivery path or just operate it.

## What is really being tested

Do you reach for "what changed" before you reach for a theory?

Can you tell Running from Ready, and say why it matters?

Do you fix the incident, or the pipeline that allowed it?

## SAY SOMETHING LIKE

"We shipped a tag that didn't exist in the registry and the pipeline still reported green. Rollback took thirty seconds. What I actually changed was the definition of a successful deploy: it now gates on rollout status with a timeout, because "the API server accepted our YAML" and "the new version is serving traffic" are different claims. Impact was partial rather than total, and that was maxUnavailable protecting us, not a control we designed — so we alert on unavailable replicas now too."

### **WEAK**

Names the tool that broke and the button they pressed. No timeline, no cost, no follow-up.

### **SOLID**

Change, detection, diagnosis, rollback, in order, with one real number attached.

### **STRONG**

Fixes the pipeline rather than the incident, names the detection gap, volunteers the cost.

# "What is an SLO, and how did you pick the number?"

The second half of the question is the real one.

## What is really being tested

Do you know SLI, SLO and SLA apart, or use them interchangeably?

Where did the number come from? Not thin air, and not 100%.

Did anyone outside engineering agree to it?

## SAY SOMETHING LIKE

"The SLI is what we measure: the share of checkout requests that succeed under 500 ms. The SLO is our internal target, 99.5% over a rolling 30 days. We picked 99.5% from what we were already achieving, about 99.7%, and from what the business could tolerate. The SLA we offer customers is deliberately looser, so we get warning before we breach a contract. And it is not 100%, because 100% leaves no room to deploy."

### WEAK

Uses SLO and SLA interchangeably, and talks about uptime in general terms.

### SOLID

Correct SLI, SLO and SLA distinction, with a target, a window and a percentile.

### STRONG

Explains where the number came from, who signed up to it, and why it is not 100%.

# "What is an error budget for?"

A maths question on the surface and a culture question underneath.

## What is really being tested

Can you do the arithmetic out loud, from an SLO?

Is it a number you report, or a tool you make decisions with?

What happens when it runs out? Policy, or vibes?

## SAY SOMETHING LIKE

"If the SLO is 99.5% over 30 days and we serve two million checkout requests, then ten thousand of them are allowed to fail or be slow. That allowance is the error budget. It is not a target to hit, it is a currency we spend on risk. While there is budget left the team ships. When it is burning fast we had already agreed to stop feature work and spend the sprint on reliability. That agreement is the point: it turns an argument about shipping into arithmetic."

## WEAK

Calls it the allowed downtime and stops there.

## SOLID

Does the arithmetic correctly from an SLO and describes spending it on risk.

## STRONG

Describes the policy agreed in advance for when it burns, and who is in that room.

# "A pod says Running but users report errors."

The trap question. It tests whether you trust the platform over the customer.

## What is really being tested

Do you know what Running actually promises? Very little.

Readiness or liveness, and what does each one gate?

Can you work a dependency chain without guessing?

## SAY SOMETHING LIKE

"Running only means the container has not exited. If the readiness probe just pings an HTTP port, the pod stays in the service endpoints and keeps taking traffic while every database call times out. So I confirm the user impact in the app itself, check whether it is one pod or all of them, then look at the dependency the service needs. Then I read one failing trace end to end and open the log line for that trace id. The fix is usually two things: whatever broke, and a readiness probe that tests the real dependency."

### WEAK

Restarts the pod. That is the whole answer.

### SOLID

Separates Running from Ready, then checks dependencies and logs rather than guessing.

### STRONG

Knows readiness gates endpoint membership, works metrics to trace to log, and names the probe as the durable fix.

# Before you leave the lab

What to keep, and the one sentence worth remembering.

## Keep these artefacts

- ☐ Your baseline numbers
- ☐ Screenshots of each monitor firing
- ☐ One trace that starts in the browser
- ☐ A postmortem for at least one scenario
- ☐ Your error-budget calculation
- ☐ One incident told from both chairs

## The whole course in one line

We monitor so we know when real users cannot do the thing they came to do.

## Practise out loud

Pair up. One asks a question from the previous slides, the other answers in ninety seconds using the five beats. Then swap chairs — literally: answer it again as the other role.

Do not forget: `./scripts/teardown.sh` before you close the laptop.